MapReduce and Scalability

Paper: “MapReduce: Simplified Data Processing on Large Clusters” by Jeffrey Dean and Sanjay Ghemawat, Google (Communications of the ACM, 2008)

Utkarsh Sharma · CS 6650 Building Scalable Systems · Midterm Mastery Part 1

Code and data: github.com/utkarshsharma19/Building-Scalable-Systems-CS6650

The problem and the idea

The problem

Google ran hundreds of jobs over huge data, like crawled web pages and request logs. Each job was simple, but it was buried in code for splitting work, moving data and handling crashes (Section 1).

The idea

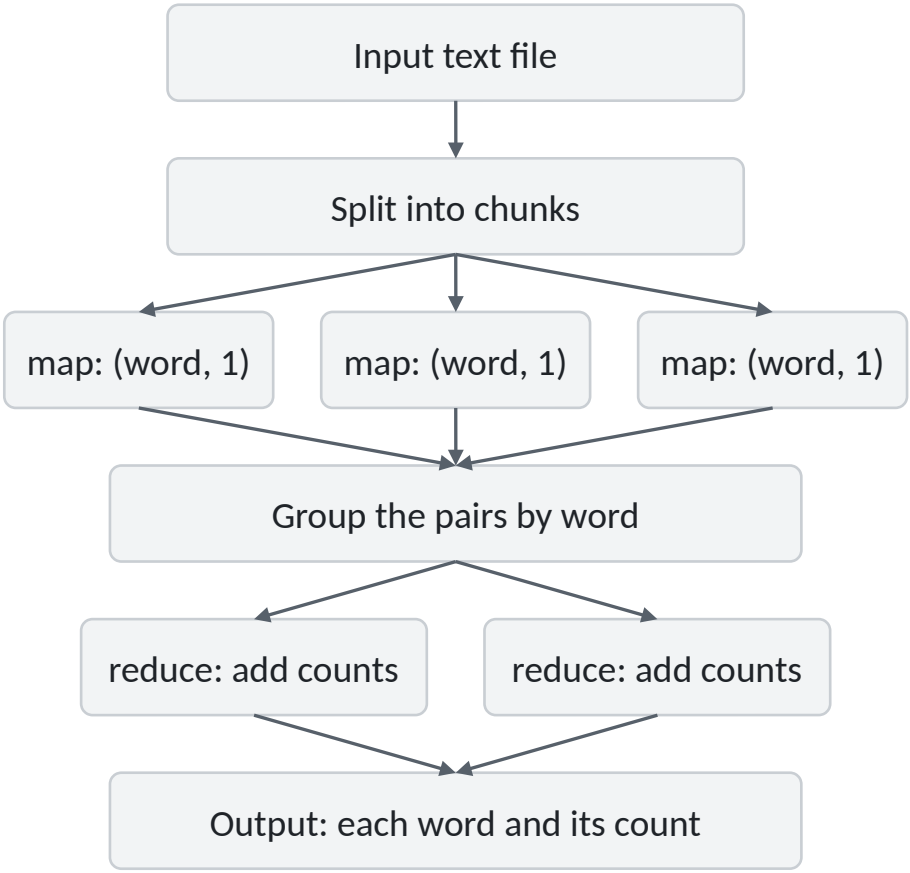
Write two functions. Map turns each record into key/value pairs. Reduce combines all the values that share a key. A master hands out the tasks, and the library does the rest (Sections 2 and 3).

Why it works

Map and reduce are deterministic and share nothing, so the system can run them in parallel and simply run them again after a failure (Section 1).

The payoff

About 100,000 jobs and over 20 PB of data a day at Google (abstract). One indexing phase shrank from about 3,800 to 700 lines of C++ (Section 6.1).



Word count: the paper's own example (Section 2.1) and my HW4 job

Key insights of the paper

The paper ends with three lessons: keep the model restricted, save the network, and run slow work twice.

1 Re-run failed work Section 3.3

A failed task simply runs again on another machine, which is safe because map and reduce are deterministic. When 80 machines at a time went offline for minutes, the job still finished.

2 Send the work to the data Section 3.4

Each map task runs on a machine that already stores its input. In the sort test, input was read at up to 13 GB/s, faster than the shuffle, because most came from local disk.

3 Run slow tasks twice Section 3.6

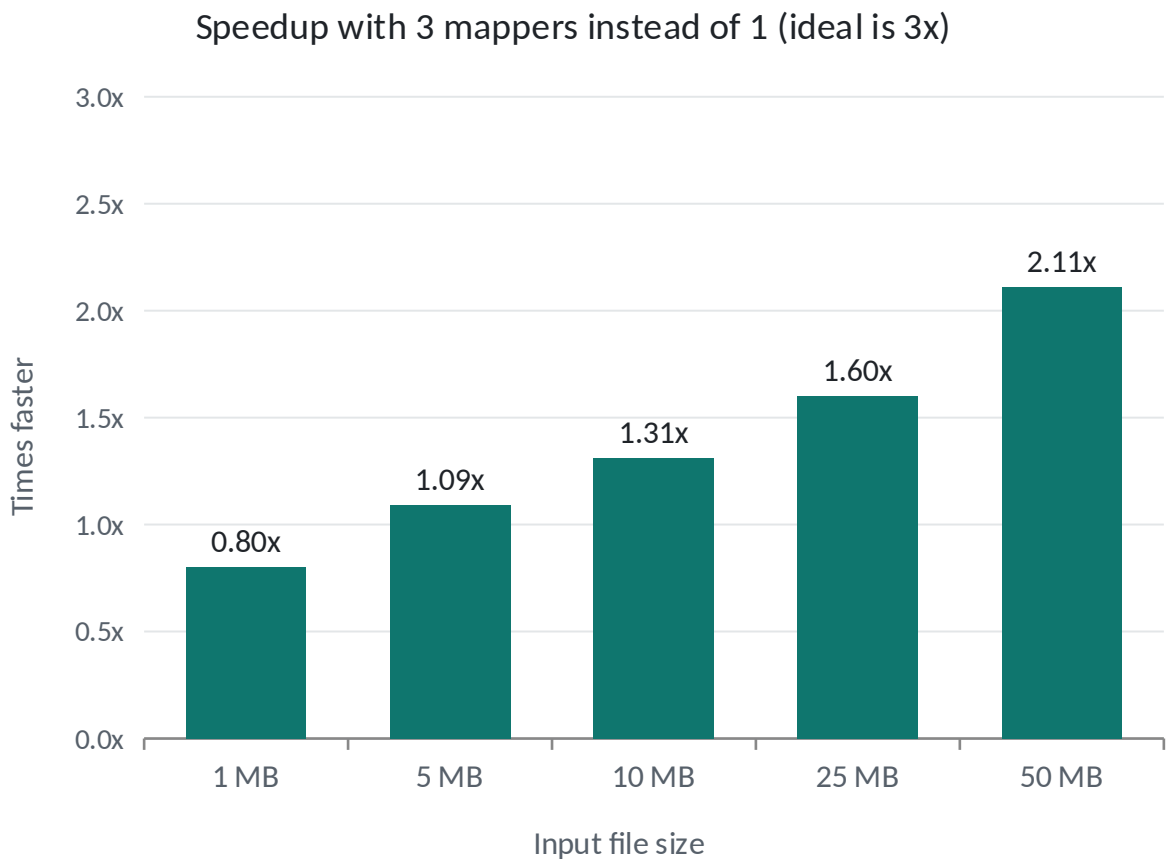
Near the end, the master starts backup copies of the last running tasks and keeps whichever finishes first. It costs a few percent more resources; without it, sort took 44% longer.

4 Many small tasks Section 3.5

Use far more tasks than machines: often 200,000 map and 5,000 reduce tasks on 2,000 machines. Fast machines take more tasks, and a dead machine's work spreads across the rest.

Source: Dean and Ghemawat, Communications of the ACM 51(1), 2008. Sections 3.3–3.6 and 5.3 (sort test).

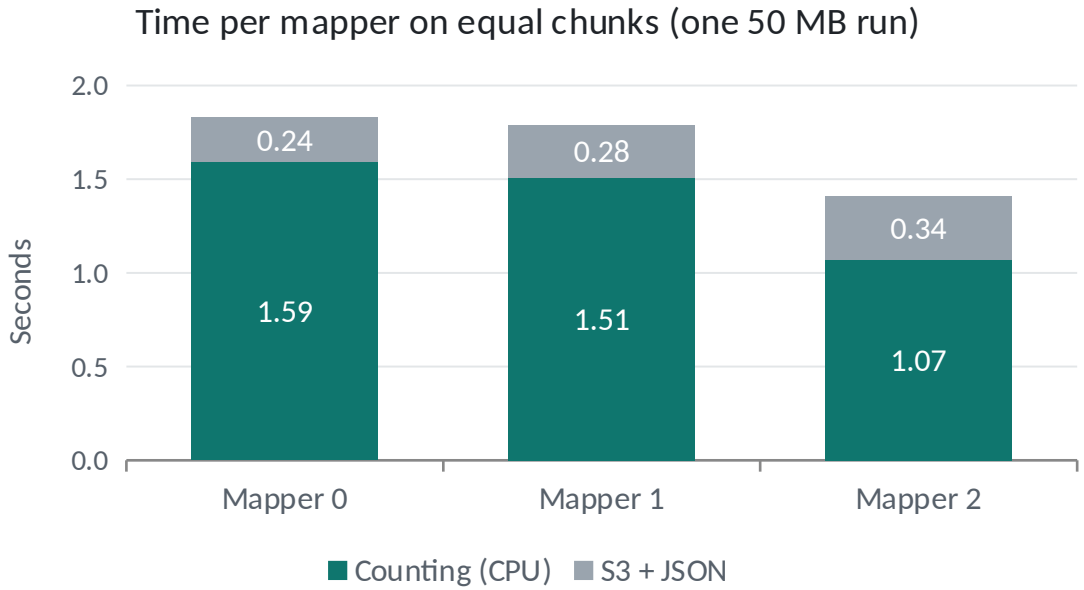
HW4: more mappers helped, but not 3x

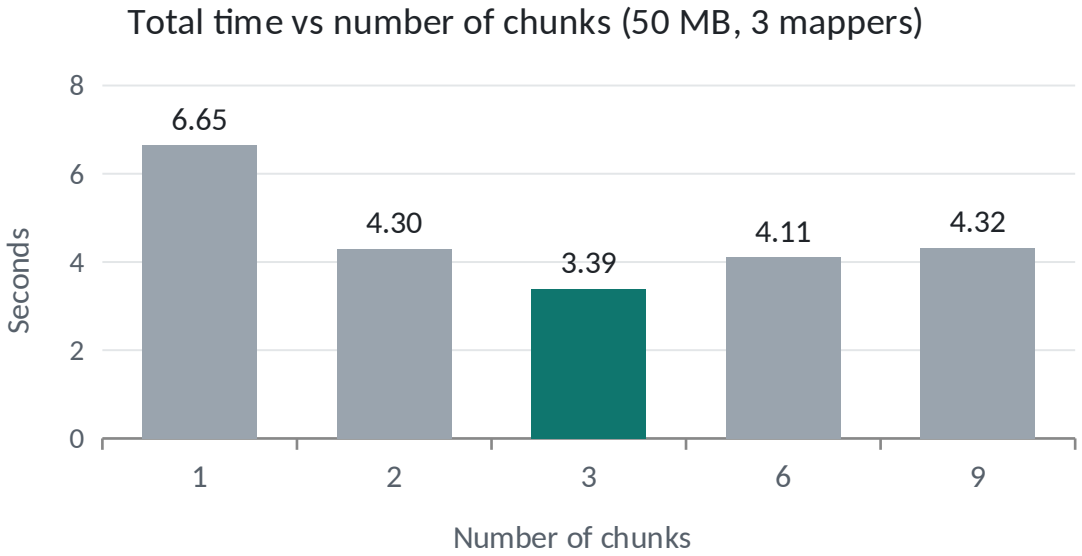


- **Small files got slower (0.80x at 1 MB).** Counting 1 MB takes 0.012 s on my laptop, but the job took 0.55 s, so almost all of it was S3 and HTTP overhead. The paper's grep job also spent about 60 of its 150 seconds starting up (Section 5.2).
- **Big files got faster, but not 3x.** At 50 MB the map step was 2.9x faster (5.40 s → 1.83 s), but the whole job was only 2.11x faster.
- **Amdahl's Law explains the gap.** In the 1-mapper run, split and reduce took 1.72 of 7.12 s (24%), and each still runs on one machine. That predicts 1 / (0.24 + 0.76/3) = 2.0x, close to my 2.11x, and a ceiling of 1 / 0.24 = 4.1x.

Data: hw04/part3/results/experiments.json. Word count on ECS Fargate with S3, median of 3 runs.

HW4: stragglers and task size showed up too





Stragglers Section 3.6

The chunks were equal (within 4 bytes), but counting took 1.07 s on one mapper and 1.59 s on another, while the S3 and JSON work took 0.24–0.34 s on all three. So the gap was CPU time, a straggler cause the paper names. Backup tasks are its fix.

Task size Section 3.5

3 chunks of about 17 MB was fastest, inside the paper's 16–64 MB range. At 9 chunks of about 6 MB, every step slowed: split +0.31 s, map +0.37 s, reduce +0.26 s. My script gave each mapper a fixed share, so small tasks added cost but no load balancing.

Data: hw04/part3/results/experiments.json (mapper times from one run; chunk test is the median of 3). Chunk handout: hw04/part3/scripts/mapreduce.py.

Question 2: How does it connect to what we covered in the course?

The same ideas in HW2, HW3 and HW6

HW3

Concurrency and Amdahl's Law

A plain Go map crashed on every run when 50 goroutines wrote to it. MapReduce avoids shared data: each task writes only its own private files (Section 3.3). Scaling also fell short: 4 Locust workers gave 2.8x, not 4x (4,238 → 11,788 requests/s), with all 4 CPUs busy.

HW6

Fault tolerance and load balancing

I stopped 3 of 4 tasks behind the load balancer: it recovered in 67 s with 0 failed requests, like the master re-running a dead worker's tasks. Load split evenly: about 717 requests/s per task on 2 tasks and 470 on 3. Core test: 1.68 million requests, 0 failures.

HW2

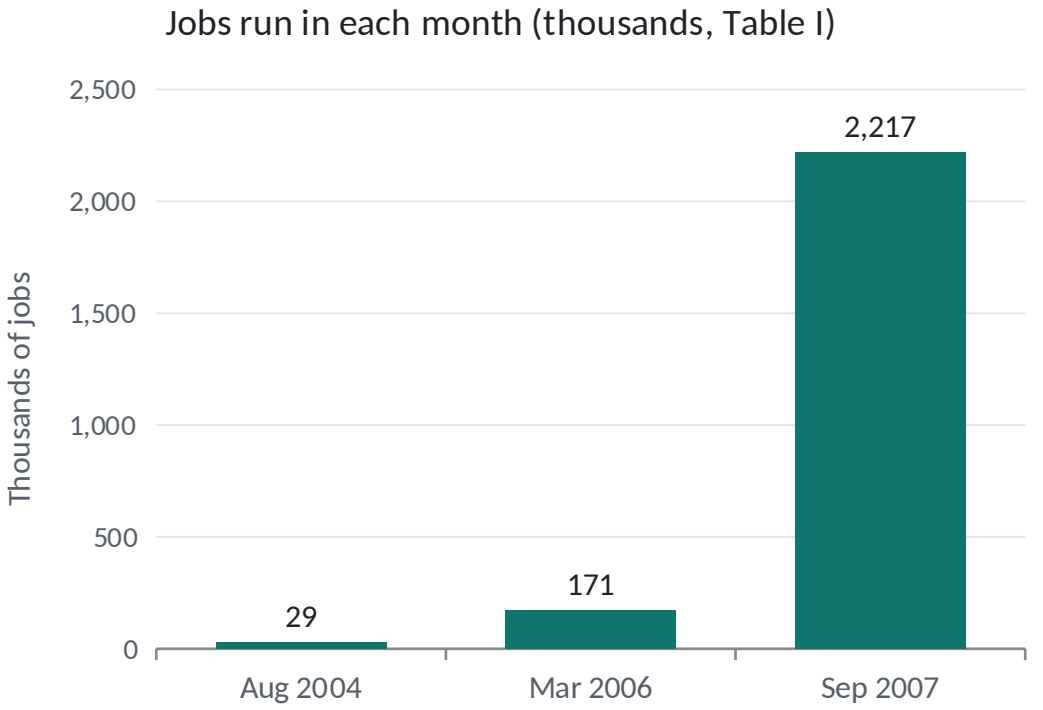
Shared state

After one POST, one server had 4 albums and the other still had 3, because each kept its data in its own memory. The paper keeps final output in a shared file system so it survives failures (Section 3.3), and my HW4 kept all task output in S3.

Data: hw02/README.md, hw03/README.md and the HW6 report (hw06/results/HW6-brief-preview.pdf) in my repo.

Why MapReduce scales

- **Many cheap machines.** Google often ran 200,000 map tasks on 2,000 commodity PCs (Sections 3 and 3.5). In HW6, 2–3 small tasks matched one 2x bigger task: about 1,450 vs 1,488 requests/s.
- **Failures are expected.** Re-running work hides them (Section 3.3). In HW6, losing 3 of 4 tasks meant 67 s of recovery and 0 failed requests.
- **Less data on the network.** Read input locally and combine early (Sections 3.4 and 4). My HW4 mappers sent at most 12,374 word counts per chunk, not about 3.2 million (word, 1) pairs.



Same table, same months: average machines per job 157 → 268 → 394; average job time 634 s → 874 s → 395 s.

Sources: paper Table I on page 112 (a subset of Google's jobs) and Sections 3–4; hw04 experiments.json and wc.go; HW6 report.

Where it stops: the paper and my data agree

Limit	What the paper shows	What I measured
Startup cost	Grep took about 150 s, and about 60 s of that was startup (Section 5.2)	HW4, 1 MB file: 3 mappers were slower than 1 (0.80x)
Slow tasks	Sort took 44% longer with backup tasks turned off (Section 3.6)	HW4: equal chunks took 1.42 s to 1.83 s, and the job waits for the slowest
One coordinator	The master keeps about 1 byte per map/reduce task pair: about 1 GB at 200,000 × 5,000 tasks (Section 3.5)	HW4: one splitter and one reducer took 24% of the time, so 4.1x at most
Small data	Its test jobs each processed about 1 TB (Section 5)	My laptop counted 50 MB in 0.58 s; my 3-mapper cluster took 3.33 s
Batch only	The average job took 395 s in September 2007 (Table I)	My HW6 API answered in 60 ms at p95. A 395 s batch job can't serve that.

Sources: paper Sections 3.5, 3.6, 5, 5.2 and Table I; hw04 experiments.json; HW6 report (p95 = 60 ms in the 60-user test).

My answers to the three questions

1

Key insights

Keep map and reduce simple and deterministic, and the system can split the work, re-run failures and run slow tasks twice. Without backup tasks, sort took 44% longer.

2

Links to the course

Amdahl's Law, stragglers, fault tolerance and shared state all showed up in my numbers: 2.11x instead of 3x (HW4), 0 failed requests (HW6), a crashing map (HW3) and servers that disagreed (HW2).

3

Scalability

It scales out on cheap machines and treats failure as normal: jobs in a month grew from 29,000 to 2.2 million (Table I). The limits are startup cost, serial parts and slow tasks.

Thank you. Questions?